UMN
UNIVERSITAS
MULTIMEDIA
NUSANTARA

IF 451
Advance Web Programming

Meet13 - Fullstack Mini Project Case Study

Wirawan Istiono, S.Kom., M.Kom

Meet 11

Objective:

Sub-CLO0712 - Students are able to build basic web programming and feature in Node JS (C6)

Sub-CLO0714 - Students are able to implement Node JS Framework to build Web application (C3, C6)

OUTLINE

- *Backend: CRUD API + file upload + login.*
- *Frontend: ReactJS interface to manage students*

Backend: CRUD API + file upload + login.

Create Database and Table Login

Create database using CLI Command or phpMyAdmin, or you can use your previous database, and then create Table Login, with detail like the sample below.

```
CREATE TABLE users (
    id int AUTO_INCREMENT PRIMARY KEY,
    username varchar(100) UNIQUE,
    password varchar(100),
    photo varchar(250)
);
```

And add at least 2 users into **table users**, like the sample below

id	username	password	photo
1	wirawan	123	
2	budi	321	

Prepare Folder Structure

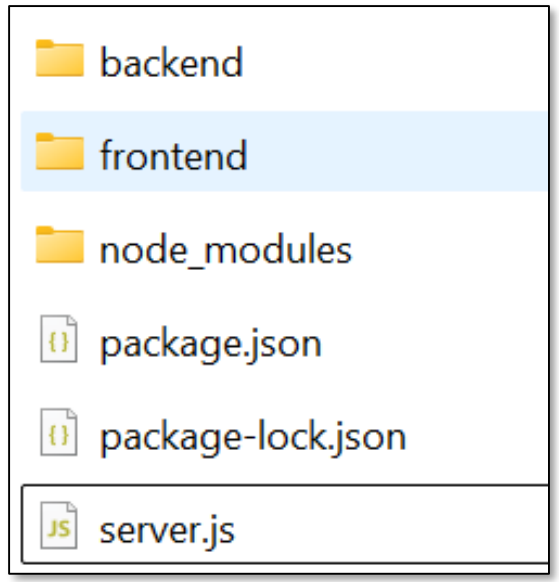
In this sample project, we'll create **Login module** and **Ms_profile** module. To do that, create folder and files like the sample below (Don't forget to install Express):

```
project_folder
|
|- backend
   |- koneksi.js
   |- login_routes.js
   |- profile.js
|- frontend
   |- public
        |- style.css
   |- views
        |- login.html
        |- profile.html
|- server.js
```

Command to Install Express:

npm init –y

npm install express



Create connection to database in koneksi.js file

Write the sample code beside at koneksi.js to create connection database:

Don't forget to install mysql, with command

npm install mysql2

To test the connection, type the command below

node backend\koneksi.js

```
PS E:\UMN\IF451 -
koneksi berhasil
```

```
const mysql = require("mysql2");

const db = mysql.createConnection({
    host:"localhost",
    user:"root",
    password:"",
    database:"dbumn2"
});

db.connect(err=> {
    if(err) {
        console.log("ada error: " + err);
    } else {
        console.log("koneksi berhasil");
    }
});

module.exports = db;
```

Create Form HTML for Login in login.html

```
<form action="/login_proses" method="POST" >
    Username: <input type="text" name="txtUsername" ><br>
    Password: <input type="password" name="txtPass" ><br>
    <input type="submit" value="Login" >
</form>
```

Create Routes to show login.html via server.js

Write the code beside in the **login_routes.js**

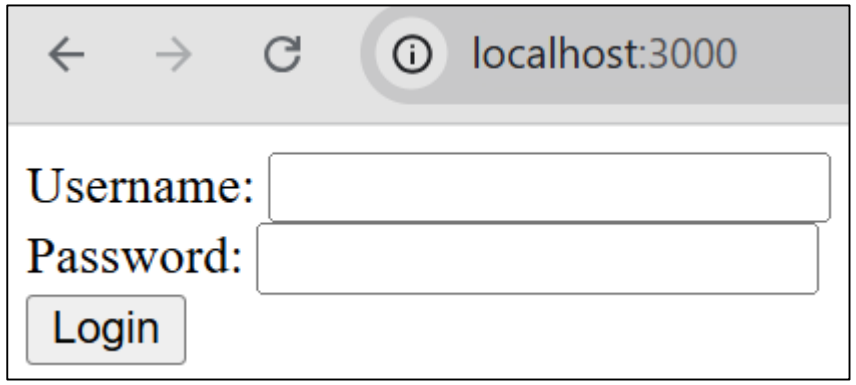
```
const express = require("express");
const path = require("path");

const router = express.Router();
router.get('/', (req , res)=>{
   res.sendFile(path.join(__dirname, "../frontend/login.html"));
})

module.exports = router;
```

Call **login_routes** from server.js

Write the code beside from server.js

After write the code beside, run node.js with the command

node server.js




```
const express = require("express");
const loginRoutes = require("./backend/login_routes");
const bodyParser = require("body-parser");
const app = express();

app.use(bodyParser.urlencoded({ extended: true }));
app.use(bodyParser.json());

app.use("/", loginRoutes);

app.listen(3000, () => {
  console.log("Server berjalan di http://localhost:3000");
});
```

Create login_cek.js

- Because in the form HTML code have `<form action="/login_proses" method="POST" >`. It means, when your click button submit, the page will redirect to login_proses and send parameter with POST datatype.
- Create **login_cek.js** file in the **backend folder**, and write the code beside
- And edit the **login_routes.js** file like the sample code beside to call and send parameter to **login_cek.js**
- After that, try to input and send data via login.html and see the result in console.

```
const kon = require("./koneksi");
const cekUserLogin = (req, res) => {
    const uname = req.body.txtUsername;
    const pass = req.body.txtPass;
    console.log("Username: " + uname);
    console.log("Password: " + pass);
    res.end();
};
module.exports = cekUserLogin;
```

```
…
const loginProsesCek = require("./login_cek");

….
router.post("/login_proses", (req, res)=> {
    loginProsesCek(req, res);
});

module.exports = router;
```

Edit **login_cek.js** to login check from database
After that, see the result in console or your browser

```
const kon = require("./koneksi");

const cekUserLogin = (req, res) => {
    const uname = req.body.txtUsername;
    const pass = req.body.txtPass;

    const q = "SELECT id FROM users WHERE username=? AND password=?";
    kon.query(q, [uname, pass], (err, results) => {
        if(err) {
            console.log("ada err: " + err);
        } else {
            if(results.length>0) {
                res.write("Login Berhasil");
            } else {
                res.write("Login Gagal");
            }
            res.end();
        }
    });
};

module.exports = cekUserLogin;
```

Save username into Session (1/2)

Sessions in Express are used to temporarily store user data on the server after a user logs in. Sessions allow you to recognize users without having to re-login each time they change pages.

To use session, install session with command:

npm install express-session

Try to edit or add the script in the **server.js** beside like the sample beside

```
….
const session = require("express-session");
const app = express();

app.use(bodyParser.urlencoded({ extended: true }));
app.use(bodyParser.json());

app.use(session({
  secret: "secret_key_umn",
  resave: false,
  saveUninitialized: true,
  cookie: { secure: false }
}));
…..
```

Save username into Session (2/2)

1. And then edit **login_cek.js** like sample beside.

```
….
  if(results.length>0) {
     req.session.username = uname;
     res.redirect("/profile");
  } else {
     res.write("Login Gagal");
  }
  res.end();
….
```

Edit login_routes for Profile

1. Add or update **login_routes.js** like the sample code beside
2. Edit or add the sample code below in **server.js** file
3. Try to login via login.html form


```
const express = require("express");
const path = require("path");
const loginProsesCek = require("./login_cek");

const router = express.Router();
router.get('/', (req , res)=>{
   res.sendFile(path.join(__dirname, "../frontend/login.html"));
});

router.post("/login_proses", (req, res)=> {
    loginProsesCek(req, res);
});

router.get("/profile", (req, res)=> {
    res.send("Ini halaman profile " + req.session.username);
})

module.exports = router;
```

REFERENCES

- B. Lawson and J. Encinas, Node.js Web Development, 6th ed., Packt Publishing, 2023
- R. Raj, Learning Node.js Development, Packt Publishing, 2018.
- E. Cantelon, M. Harter, T. Holowaychuk, and N. Rajlich, Node.js in Action, 2nd ed., Manning Publications, 2017.
- Maulana, a., bau, r.t.r., munawar, z., setiawan, r., aisa, s., istiono, w., irfan, a., pratama, y.a., ramadhany, e.d., pomalingo, s. And permana, a.a., 2023. *Pemrograman web 101: memahami dasar-dasar untuk mengembangkan situs web*. Get press indonesia.
- The Node.js Foundation, "Node.js Documentation," [Online]. Available: https://nodejs.org/en/docs/. [Accessed: Jun. 5, 2025].
- Express.js Team, "Express.js Documentation," [Online]. Available: https://expressjs.com/. [Accessed: Jun. 5, 2025].

Visi

Menjadi Program Studi Strata Satu Informatika **unggulan** yang menghasilkan lulusan **berwawasan internasional** yang **kompeten** di bidang Ilmu Komputer (*Computer Science*), **berjiwa wirausaha** dan **berbudi pekerti luhur**.

INFORMATIKA
UMN
UNIVERSITAS MULTIMEDIA NUSANTARA

Misi

1. Menyelenggarakan pembelajaran dengan teknologi dan kurikulum terbaik serta didukung tenaga pengajar profesional.
2. Melaksanakan kegiatan penelitian di bidang Informatika untuk memajukan ilmu dan teknologi Informatika.
3. Melaksanakan kegiatan pengabdian kepada masyarakat berbasis ilmu dan teknologi Informatika dalam rangka mengamalkan ilmu dan teknologi Informatika.